# 83. Адресная арифметика и индексация массивов

Язык C++ позволяет выполнять целочисленные операции сложения/вычитания с **указателями**. Если `ptr` указывает на целое число, то `ptr + 1` является адресом следующего целочисленного значения в памяти после `ptr`. `ptr - 1` — это адрес предыдущего целочисленного значения (перед `ptr`).

**Оглавление:**


1. **Адресная арифметика**
2. **Расположение элементов массива в памяти**
3. **Индексация массивов**
4. **Использование указателя для итерации по массиву**


## Адресная арифметика

Обратите внимание, `ptr + 1` не возвращает следующий *любой адрес памяти*, который находится сразу после `ptr`, но он возвращает *адрес памяти следующего объекта, тип которого совпадает* с типом значения, на которое указывает `ptr`. Если `ptr` указывает на адрес памяти целочисленного значения (размер которого 4 байта), то `ptr + 3` будет возвращать адрес памяти третьего целочисленного значения после `ptr`. Если `ptr` указывает на адрес памяти значения **типа char**, то `ptr + 3` будет возвращать адрес памяти третьего значения типа char после `ptr`.

При вычислении результата выражения **адресной арифметики** (или ***«арифметики с указателями»***) компилятор всегда умножает целочисленный операнд на размер объекта, на который указывает указатель. Например:

```
#include <iostream>

int main()
{
    int value = 8;
    int *ptr = &value;

    std::cout << ptr << '\n';
    std::cout << ptr+1 << '\n';
    std::cout << ptr+2 << '\n';
    std::cout << ptr+3 << '\n';

    return 0;
}
```

Результат на моем компьютере:

```
002CF9A4
002CF9A8
002CF9AC
002CF9B0
```

Как вы можете видеть, каждый последующий адрес увеличивается на 4. Это связано с тем, что размер типа int на моем компьютере составляет 4 байта.

Та же программа, но с использованием типа short вместо типа int:

```
#include <iostream>

int main()
{
    short value = 8;
    short *ptr = &value;

    std::cout << ptr << '\n';
    std::cout << ptr+1 << '\n';
    std::cout << ptr+2 << '\n';
    std::cout << ptr+3 << '\n';

    return 0;
}
```

Результат:

```
002BFA20
002BFA22
002BFA24
002BFA26
```

Поскольку тип short занимает 2 байта, то каждый следующий адрес больше предыдущего на 2.

## Расположение элементов массива в памяти

Используя оператор адреса `&`, мы можем легко определить, что элементы массива расположены в памяти последовательно. То есть, элементы 0, 1, 2 и т.д. размещены рядом (друг за другом):

```
#include <iostream>

int main()
{
    int array[] = { 7, 8, 2, 4, 5 };

    std::cout << "Element 0 is at address: " << &array[0] << '\n';
    std::cout << "Element 1 is at address: " << &array[1] << '\n';
    std::cout << "Element 2 is at address: " << &array[2] << '\n';
    std::cout << "Element 3 is at address: " << &array[3] << '\n';

    return 0;
}
```

Результат на моем компьютере:

```
Element 0 is at address: 002CF6F4
Element 1 is at address: 002CF6F8
Element 2 is at address: 002CF6FC
Element 3 is at address: 002CF700
```

Обратите внимание, каждый из этих адресов по отдельности занимает 4 байта, как и размер типа int на моем компьютере.

## Индексация массивов

Мы уже знаем, что элементы массива расположены в памяти последовательно. Мы знаем, что **фиксированный массив** может распадаться на указатель, который указывает на первый элемент (элемент под индексом 0) массива.

Также мы уже знаем, что добавление единицы к указателю возвращает адрес памяти следующего объекта этого же типа данных.

Следовательно, можно предположить, что добавление единицы к идентификатору массива приведет к возврату адреса памяти второго элемента (элемента под индексом 1) массива. Проверим на практике:

```
#include <iostream>

int main()
{
    int array [5] = { 7, 8, 2, 4, 5 };

    std::cout << &array[1] << '\n'; // выведется адрес памяти элемента под индексом 1
    std::cout << array+1 << '\n'; // выведется адрес памяти указателя на массив + 1

    std::cout << array[1] << '\n'; // выведется 8
    std::cout << *(array+1) << '\n'; // выведется 8 (обратите внимание на скобки, они здесь обязательны)

    return 0;
}
```

При разыменовании результата выражения адресной арифметики скобки необходимы для соблюдения **приоритета операций**, поскольку оператор `*` имеет более высокий приоритет, чем оператор `+`.

Результат выполнения программы на моем компьютере:

```
001AFE74
001AFE74
8
8
```

Оказывается, когда компилятор видит оператор индекса `[]`, он, на самом деле, конвертирует его в указатель с операцией сложения и разыменования! То есть, `array[n]` — это то же самое, что и `*(array + n)`, где `n` является целочисленным значением. Оператор индекса `[]` используется в целях удобства, чтобы не нужно было всегда помнить о скобках.

## Использование указателя для итерации по массиву

Мы можем использовать указатели и адресную арифметику для выполнения итераций по массиву. Хотя обычно это не делается (использование оператора индекса, как правило, читабельнее и менее подвержено ошибкам), следующий пример показывает, что это возможно:

```
#include <iostream>

int main()
{
    const int arrayLength = 9;
    char name[arrayLength] = "Jonathan";
    int numVowels(0);
    for (char *ptr = name; ptr < name + arrayLength; ++ptr)
    {
        switch (*ptr)
        {
        case 'A':
        case 'a':
        case 'E':
        case 'e':
        case 'I':
        case 'i':
        case 'O':
        case 'o':
        case 'U':
        case 'u':
            ++numVowels;
        }
    }

    std::cout << name << " has " << numVowels << " vowels.\n";

    return 0;
}
```

Как это работает? Программа использует указатель для *прогона* каждого элемента массива поочередно. Помните, что массив распадается в указатель на первый элемент массива? Поэтому, присвоив `name` для `ptr`, сам `ptr` стал указывать на первый элемент массива. Каждый элемент разыменовывается с помощью выражения **switch**, и, если текущий элемент массива является гласной буквой, то `numVowels` увеличивается. Для перемещения указателя на следующий символ (элемент) массива в **цикле for** используется оператор `++`. Работа цикла завершится, когда все символы будут проверены.

Результат выполнения программы:

```
Jonathan has 3 vowels.
```